

SOEN 341 — Software Process

CareerConnect

AI Usage Log — Sprint 1

Liam

Fall 2026

Running log of every AI interaction, kept in the format required by the Sprint 1 spec ("AI Usage Log Requirements"). At the end of each sprint this gets exported to PDF and uploaded to my personal subfolder in the repo's AI log directory (currently named **AI_Logs/** in the repo; the spec says **AI_Log/**).

Sprint 1

Entry 1.1 — Development environment setup guidance: tool check (VS Code, Git on macOS)

- **Date:** 2026-09-26
- **Purpose of AI Use:** Research and information gathering / learning — step-by-step guidance for setting up a JavaScript + HTML development environment on macOS and connecting it to the team's GitHub repository.
- **Chat Link or Prompt/Response:** Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt (summary): "I am a COEN student... web development is completely new to me... start from setting up my VS Code with the languages we are using (JavaScript and HTML) and for it to be set up with our GitHub for the project. Make steps very thorough and only move on once I've completed the previous one."
- **AI-Suggested Content:** A 7-step setup roadmap (check tools → install Node.js → configure Git identity → authenticate with GitHub → clone repo → VS Code extensions → install dependencies & run). Step 1: Terminal commands to check installed tools (`git --version`, `node --version`, `npm --version`, `code --version`, `uname -m`) and how to add the VS Code `code` command to PATH.
- **Outcome:** Git was already installed (via Xcode). `code` command was missing and was installed through VS Code's Command Palette; verified working. Node.js and npm were not installed.
- **Validation:** Ran each command myself and confirmed the output in Terminal.
- **Decision:** Accepted
- **Reflection:** By using ai as a learning tool I grasped a better understanding of how to setup plugins and git through the vscode terminal
- **Responsible Person:** Liam

Entry 1.2 — Installing Node.js and npm on macOS

- **Date:** 2026-09-26
- **Purpose of AI Use:** Research and information gathering / learning — how to install the JavaScript runtime needed to run the project locally.
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1.
 - Prompt: "I had git and I installed the code command path and it worked, I did not have anything else." Follow-up prompt: "Can you make the instructions a little less wordy."
- **AI-Suggested Content:** Explanation of what Node.js/npm are; install the LTS version via the official macOS .pkg installer from nodejs.org; check whether the team uses a specific Node version; verify with `node --version`, `npm --version`, and `node -e "console.log('Hello from JavaScript!')"`. Instructions were condensed on request.
- **Outcome:** Installed Node.js v24.21.0 and npm 11.19.0; test script printed "Hello from JavaScript!".
- **Validation:** Verified versions and ran the test command in Terminal.
- **Decision:** Modified before use (asked the AI to shorten the instructions). Accepted
- **Reflection:** I became more familiar npm in terms of what it does and why I need it as well as become more familiar with installing plugins in vscode
- **Responsible Person:** Liam

Entry 1.3 — Configuring Git identity

- **Date:** 2026-09-26
- **Purpose of AI Use:** Research and information gathering / learning — configuring Git so commits are attributed to my GitHub account.
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "The things that showed are as follow: v24.21.0, 11.19.0, Hello from JavaScript!"
Follow-up: "What do I replace "code --wait" with" (AI clarified that only name and email are replaced; the other lines are typed as-is).
- **AI-Suggested Content:** `git config --global` commands to set `user.name`, `user.email` (matching the GitHub account email), default branch `main`, and VS Code as Git's editor (`code --wait`); verify with `git config --global --list`.
- **Outcome:** All four settings applied and confirmed in `git config --global --list` output.
- **Validation:** Checked the `--list` output against the values I entered and my GitHub email settings.
- **Decision:** Accepted
- **Reflection:** I never had to do this because I used Clion previously and I do not remember how I did it but the git implementation just seemed to work
- **Responsible Person:** Liam

Entry 1.4 — Connecting my Mac to GitHub with an SSH key

- **Date:** 2026-09-26
- **Purpose of AI Use:** Research and information gathering / learning — authenticating Git on my Mac with GitHub so I can clone and push to the team repo.
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "It worked and all of the information I entered showed". Follow-up: "When I get to this step, `ssh-add --apple-use-keychain ~/.ssh/id_ed25519`, do I enter this in vs code terminal or the regular terminal" (AI: regular Terminal; either works).
- **AI-Suggested Content:** Generate an ed25519 SSH key (`ssh-keygen`), store the passphrase in the macOS Keychain via `~/.ssh/config` and `ssh-add --apple-use-keychain`, copy the public key with `pbcopy`, add it under GitHub Settings → SSH and GPG keys, and test with `ssh -T git@github.com`.
- **Outcome:** SSH key created, added to GitHub, and connection test succeeded.
- **Validation:** `ssh -T git@github.com` authenticated me by my GitHub username.
- **Decision:** Accepted.
- **Reflection:** I had asked ai to guide me through this task as a reassurance as I was following the steps listed on the official GitHub guide.
- **Responsible Person:** Liam

Entry 1.5 — Cloning the team repository and opening it in VS Code

- **Date:** 2026-09-26
- **Purpose of AI Use:** Research and information gathering / learning — downloading the team's GitHub repo to my Mac.
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "The key setup worked on GitHub, I am ready to carry on." Follow-ups: asked which link to copy since the GitHub project showed two repos (AI explained the Languages bar vs. two separate repos; I confirmed only one repo is used, the other is a demo).
- **AI-Suggested Content:** Create a `~/Projects` folder, copy the repo's SSH URL from GitHub, `git clone`, open with `code .`, trust the workspace, and inspect with `ls`, `git status`, and `git branch -a`.
- **Outcome:** Repo cloned successfully. Structure: `client/`, `server/`, `documentation/`, `minutes/`, `AI_Logs/`, `README.md`, `userStories.md`, root `package.json/package-lock.json`, and a committed `node_modules/`. Default branch is `master`; working tree clean.
- **Validation:** `git status` reported "up to date with origin/master... working tree clean"; files visible in VS Code.
- **Decision:** Modified before use (cloned only the real project repo, not the demo repo).
- **Reflection:** Prior to this project I had very little experience with working on GitHub with teammates in the same repository so I followed the ai instructions to make sure I did not do damage to my own work or my teammates' work
- **Responsible Person:** Liam

Entry 1.6 — VS Code extensions for JavaScript/HTML and inspecting the project setup

- **Date:** 2026-09-27
- **Purpose of AI Use:** Research and information gathering / learning — configuring VS Code for JavaScript/HTML development; AI review of repo structure.
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: pasted output of `ls`, `git status`, `git branch -a`; then pasted `package.json`, `ls client server`, and `git ls-files node_modules` output.
- **AI-Suggested Content:** Install ESLint, Prettier, Live Server, and GitHub Pull Requests extensions via `code --install-extension`. AI review of the repo found: (1) `node_modules/` is committed to Git (confirmed by `git ls-files`) and should be removed and added to `.gitignore`; (2) log folder named `AI_Logs` vs. spec's `AI_Log`; (3) `package.json` lists `crypto`, `http`, and `path` as npm dependencies although they are built into Node (the npm `http` package is a placeholder) — can be removed; (4) `client/` is empty and `server/` only has `index.js`; app starts with `npm start` → `node server/index.js`. Recommended raising these with the team rather than changing them alone.
- **Outcome:** I installed the tools I needed to work on the project.
- **Validation:** I verified that all of the tools I needed were in fact installed on my machine and that I was installing what is necessary for the project
- **Decision:** Modified before use
- **Reflection:** By this point I was become pretty familiar with installing extensions but I wanted to make absolutely sure that I was doing things correctly as well as learn about what the terminal outputs I was getting meant.
- **Responsible Person:** Liam

Entry 1.7 — Integrating teammate's front-end code (planning)

- **Date:** 2026-09-27
- **Purpose of AI Use:** Project planning / process — how to add a teammate's front-end code (shared on Discord) to the repo, polish and complete it, and push it properly.
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "...my front end teammate sent me the code that he worked on so far on discord and asked me to add it to the code and polish it and complete it and when I was done, push it onto the repo. Should I send you that code?"
- **AI-Suggested Content:** Yes, share the code plus `server/index.js`. Recommended: work on a feature branch rather than `master`; preserve teammate credit, since the rubric grades each member's contribution; open a pull request for review per the team process.
- **Outcome:** Team decided credit will be given partly during the demo and partly in commit messages.
- **Validation:** Discussed with team and confirmed that the code was free of syntax errors and that it was placed in the appropriate directory based on my teammates' workflow
- **Decision:** Modified before use (team chose its own credit approach).
- **Reflection:** I have a much better understanding how web development is formatted and how each components works with one another in bigger projects so in the future, I will likely be able to organise a project on my own
- **Responsible Person:** Liam

Entry 1.8 — Code review of teammate's login/register pages and server; committing the original code on a feature branch

- **Date:** 2026-09-27
- **Purpose of AI Use:** Code review + process guidance — review of teammate's front-end code (register.html, register.css, register.js, login.html, pasted from Discord) and `server/index.js`; how to add the original code to the repo on a branch with teammate credit.
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: pasted teammate's three code "clumps" and attached `server/index.js`, noting credit will be given in the demo and commit messages. Follow-ups: whether files go in `client/` (yes); VS Code flagged an error on the first 3 lines of `register.css` (AI confirmed it's the missing selector and gave the fix `*, ::before, ::after { box-sizing: border-box; }`, suggesting it be committed separately after the original code).
- **AI-Suggested Content:**
 - ***Plan:** commit teammate's code **exactly as sent** in a first commit crediting him (with a `Co-authored-by:` trailer), then make my polish/completion changes in separate commits so the history shows who did what.
 - ***Front-end issues found:** `register.html` links `resgister.css` (typo, so page loads unstyled); CSS begins with `{ box-sizing... }` with no selector (should be `*, ::before, ::after`); `login.js` referenced but not provided; `logo.png` missing; `login.html` missing `lang`, viewport meta; neither page has a `<title>`; password rules only shown after a failed submit; no redirect after successful registration; button not disabled during submit.
 - ***Server issues found (for back-end teammate):** no `express.static` so the server doesn't serve `client/`; no `express.json()` so JSON bodies can't be read; no `/api/register` or `/api/login` routes (front end calls them); `/upload` sends a response before checking for a file, calls `res.json` twice, and reads `req.file.file.filename` / `req.file.path` which don't exist with `memoryStorage`; `diskstorage` and `formidable` unused.
 - ***Step 7 guidance:** create branch `feature/login-register-pages`, create files in `client/`, preview with Live Server, `git add + git commit` with credit trailer.
- **Outcome:** Files created in `client/`; VS Code's CSS linter independently confirmed the selector error. The error was fixed before the first commit
- **Validation:** VS Code's built-in CSS validation flagged the same error the AI identified.
- **Decision:** Modified before use
- **Reflection:** I had not noticed that ai had flagged the same error, so I went looking for what was wrong with the syntax only to realise that I had found the same solution as the ai proposed
- **Responsible Person:** Liam (original front-end code by Amer Abou)

Entry 1.9 — Debugging: `git add client` not staging files (`client/` was a broken submodule)

- **Date:** 2026-09-27
- **Purpose of AI Use:** Debugging and problem solving.
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: after `git add client`, `git status` only listed `.DS_Store` as untracked; pasted outputs of `ls client`, `cat .gitignore`, `git check-ignore`, `git log --oneline -3`, then `git status`, `git diff --cached --stat`, `ls -a client`.
- **AI-Suggested Content:** `.DS_Store` is a macOS Finder file and should be ignored, not committed. Diagnosed from the error `fatal: Pathspec 'client/register.html' is in submodule 'client'` that `client/` is recorded in the repo as a **submodule/gitlink** (a pointer to another Git repo) rather than a normal folder — most likely because the teammate who created `client/` had a nested `.git` inside it when committing. This is why `client/` was empty after cloning and why Git ignores files placed in it. Fix: verify with `git ls-files -s client` (mode `160000`) and check for `.gitmodules`; then `git rm --cached client` and `git add client` to track it as a normal folder. Recommended warning the team.
- **Outcome:** Decided not to apply the fix myself; waiting for the teammate who set up the repository (available next morning) to handle it.
- **Validation:** I did not understand what the problem was and I was sceptical that the ai was giving the correct solution because I had discussed with one of my teammates.
- **Decision:** Deferred to be handled by the repo owner.
- **Reflection:** I opted to wait until I was certain of the solution before acting, which I believe was the correct course of action.
- **Responsible Person:** Liam

Entry 1.10 — Planning and learning: building the user registration/login back end (Node.js + Express)

- **Date:** 2026-09-27
- **Purpose of AI Use:** Learning + planning + implementation guidance — I was assigned the user login back end and have no prior web back-end experience.
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "...One of our classmates will do the html file for the Resume Upload and I will do the user login back end. Can you walk me through the steps to accomplish this while also teaching me how to do it because I have never done anything like this."
 - AI asked 3 clarifying questions; my answers: scope = register + login; storage = simple JSON file for Sprint 1 (no DB chosen yet); teaching style = mix of code-with-explanation and trying parts myself.
 - Follow-up (exercise attempt): I wrote `app.get("/api", ...)` returning "Hi, I'm Alexi" and still saw "Cannot GET /api/login". AI explained: "Cannot GET /api/login" was the expected result of B1d (that route doesn't exist yet); my exercise route used the path `/api` instead of `/api/hello`, and since Express uses the first matching route, the original `/api` route always answers first, so mine never runs. Fix: change the path to `/api/hello`, restart the server, visit that URL.
- **AI-Suggested Content:** Roadmap B1–B8: (B1) concepts + new branch `feature/login-backend` from `master` + run the existing server; (B2) read JSON request bodies with `express.json()` and serve `client/`; (B3) user storage module using a JSON file; (B4) `POST /api/register` with validation and 409 on duplicate email; (B5) password hashing with Node's built-in `crypto.scrypt`; (B6) `POST /api/login`; (B7) testing with curl and the front-end pages; (B8) commit, push, pull request. B1 content: explanation of client/server, HTTP requests/responses, routes, JSON, Express; walkthrough of existing `server/index.js`; commands `git switch master`, `git switch -c feature/login-backend`, `npm start`, visit `http://localhost:3001/api`.
- **Outcome:** Server ran; `/api` returned the hello message; `/api/login` returned "Cannot GET" as expected. Exercise route `GET /api/hello` returned `{"message": "Hi, I'm Alexi"}` after fixing the path (practice route deleted afterwards).
- **Validation:** I was completely new to web development and javascript so I opted to use ai as a learning tool in order to become for familiar with the topic and to hopefully be more confident in my abilities by the end of the proposed roadmap.
- **Decision:** Accepted
- **Reflection:** When topics seemed a little to complex, I would take more time to read the ai explanation and find other sources such as YouTube videos to better grasp the concept. But overall, the ai provided explanations I was satisfied with.
- **Responsible Person:** Liam

Entry 1.11 — B2: Middleware — reading JSON request bodies and serving the client pages

- **Date:** 2026-09-27
- **Purpose of AI Use:** Learning + implementation guidance (back end).
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "What appeared was: message "Hi, I'm Alexi"" (B1 exercise succeeded after fixing the route path).
- **AI-Suggested Content:** Explanation of **middleware** (functions that run on every request before the routes, registered with `app.use`). Experiment: write a temporary `POST /api/echo` route that returns `req.body`, test with `curl *without* express.json()` (body is missing), then add `app.use(express.json())` and see the data arrive. Add `app.use(express.static(path.join(__dirname, "..", "client")))` so the server sends the HTML/CSS/JS files in `client/`; explanation of `__dirname` and `path.join`; test by visiting `http://localhost:3001/register.html`.
- **Outcome:** Initially got "Cannot GET /register.html". After moving the two `app.use` lines (and re-saving/restarting), the register page loaded from the server. AI clarified that blank lines don't matter in JavaScript — what matters is that middleware is registered after `const app = express();` and before the routes, and that the file is saved and the server restarted after every change. `curl echo` test returned `{"youSent":{"email":"test@test.com"}}` (the trailing % is just zsh marking a missing newline).
- **Validation:** Register page loaded at `http://localhost:3001/register.html`. `curl echo` test confirmed `express.json()` parses JSON bodies.
- **Decision:** Accepted
- **Reflection:** Having the ability to test the code that is provided permits me to fact check that the commands I am given are correct and permits me to learn while doing
- **Responsible Person:** Liam

Entry 1.12 — Research: what Supabase and Vercel are, and how they change the login back-end plan

- **Date:** 2026-09-27
- **Purpose of AI Use:** Research and information gathering; system design discussion.
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "I am now realizing that we have established a method in which we will manage the back end and it is in this list (I think its Supabase and Vercel but I am not sure what those are)... HTML, CSS, JavaScript, Python, Node.js, Express.js, Supabase, Vercel, Git and GitHub"
- **AI-Suggested Content:** Supabase = hosted PostgreSQL database + built-in user authentication (sign-up/login, password hashing, sessions) + file storage (useful for resume uploads). Vercel = hosting/deployment platform that puts the site and Express server online, auto-deploying from GitHub; supports Express with zero configuration. Revised plan: replace the JSON-file user store and hand-written password hashing with **Supabase Auth** called from Express (`supabase.auth.signUp` for `/api/register`, `supabase.auth.signInWithPassword` for `/api/login`). Keys must be stored in a `.env` file excluded by `.gitignore`, never committed; only the server uses the secret key. Recommended getting access to the team's Supabase project and fixing the committed `node_modules/` + missing `.gitignore` before installing `@supabase/supabase-js`. B2 (middleware) is unaffected and can continue. The AI verified Supabase's current API key types and Vercel's Express support against official docs (supabase.com/docs/guides/getting-started/api-keys, vercel.com/docs/frameworks/backend/express).
- **Outcome:** I opted to use the Supabase for the first sprint because that is what was decided by my team
- **Validation:** I confirmed that Superbase was what we were using by looking back on previous chat logs
- **Decision:** Accepted
- **Reflection:** I felt as though after each ai prompt and explanation I was receiving; I came to understand certain components more clearly without having to refer to web sources or ai
- **Responsible Person:** Liam

Entry 1.13 — B3: Route skeletons with server-side input validation for `/api/register` and `/api/login`

- **Date:** 2026-09-27
- **Purpose of AI Use:** Learning + code generation (register route) + exercise (login route written by me).
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "This is what it says bellow the curl command:
`{"youSent":{"email":"test@test.com"}}%`"
- **AI-Suggested Content:** Revised roadmap (B3 skeletons → B4 Supabase access + .env → B5 connect → B6 register via signUp → B7 login via signInWithPassword → B8 test → B9 PR) so work can continue before Supabase access. Explained why the server must re-validate input even though the page already does (requests can be sent without the page, e.g., with curl). Provided `POST /api/register` skeleton: destructuring `req.body ?? {}`, required-field check, password rules matching `register.js` (10–20 chars, a digit, one of `! @ # $ % ^ & ( )`), `return res.status(400).json(...)` on failure, temporary `501 Not Implemented` on success. Explained status codes 400/501, `return` to stop early, regex `.test()`. Exercise: write `POST /api/login` myself (400 if email or password missing, otherwise 501 placeholder). Tests with `curl -i` for missing fields, short password, valid input.
 - Follow-up: I shared my first attempt at `/api/login` (checked `!email`, returned 400 with key `errorrr` and message "Email is not registered to an account.", no final response) and asked how to check the password matches the email. AI review: checking the email/password match needs stored accounts, which Supabase will do in B7 — not part of this step. Issues to fix: (1) `!email` means "missing", not "not registered", so the message is misleading; (2) also check `password`; (3) typo `errorrr` → `error` (front end and back end must agree on key names); (4) no response when input is valid, so the request would hang forever — add the 501 placeholder. Security lesson for B7: on failed login, reply with a generic "Invalid email or password." (401) rather than revealing whether an email is registered (prevents account enumeration).
 - Second attempt: `if (!email || !password) → 400 { error: "Email and Password are required." }`, then `res.status(501).json(...)` placeholder. AI review: logic correct and all four issues fixed. Minor polish only: message says "Login passed" (better: "Input valid"), typo "functionaliity", missing semicolon (optional in JS but keep consistent), indentation — use VS Code "Format Selection" (⌘K ⌘F) to tidy only my code without reformatting teammates' code.
- **Outcome:** `/api/login` skeleton written by me after two iterations with AI feedback. B3c tests all behaved as expected: register with missing fields → 400 "Name, email and password are required."; short password → 400 "Password must be 10 to 20 characters."; valid input → 501 placeholder; login with missing password → 400 "Email and Password are required." AI noted my custom 501 message ("Acount Regestration successful...") has typos and claims success before an account exists — reword. Next: remove the temporary `/api/echo` route, commit only `server/index.js` (`git add server/index.js`, not `.DS_Store`) with a message matching the team's "Feat: ..." style, and push the branch as a backup. Done: removed `/api/echo`, committed `server/index.js`, and pushed `feature/login-backend` to GitHub.

- **Validation:** I was able to observe the results printed in the terminal and cross reference them with the error messages I had established in the back end of the user register
- **Decision:** Accepted
- **Reflection:** After this step, I better understood how the workflow for troubleshooting (at least in simple projects) works. I understood that I can communicate with the server and test it by observing the output of both the terminal and the local website. It was also at this point where I noticed how I was over estimating the difficulty of the project and was in fact fully capable of developing this website a little more independently.
- **Responsible Person:** Liam

Entry 1.14 — B4: Creating the Supabase project and storing keys safely (.env)

- **Date:** 2026-09-27
- **Purpose of AI Use:** Learning + setup guidance; security practices for secrets.
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "My classmate pushed his work on feature/registration-login front end branch which is only missing the resume upload page. My team does not yet have a supabase project. I will not worry about the .gitignore yet because I would like to focus on my work. I am ready for the next step."
- **AI-Suggested Content:** Since no Supabase project exists, create one myself (as the login back-end owner) and invite the team. Steps: sign in at supabase.com, create project (name, strong database password saved in a password manager, Canada region, free plan); copy the Project URL and **publishable key** (use the publishable key, not the secret key, since sign-up/login are public auth operations — least privilege). Explained environment variables and why keys go in a .env file that is never committed. Even while deferring the team-wide .gitignore, add a minimal .gitignore containing .env (and .DS_Store) on my branch to prevent leaking keys; leave the node_modules/cleanup to the team. Load .env with Node's built-in --env-file-if-exists=.env flag in the start script (no extra package; teammates without a .env can still start the server). Verify with a temporary console.log and confirm git status does **not** list .env. AI checked Supabase's current API key types against docs (supabase.com/docs/guides/getting-started/api-keys).
 - Follow-ups: asked where the start line is in package.json (AI pointed to the "scripts" section and explained JSON comma rules); accidentally pasted the JavaScript console.log line into Terminal (zsh error "unknown username") — AI explained that js/json blocks go in files and bash blocks are Terminal commands.
- **Outcome:** Supabase project created; .gitignore (.env, .DS_Store) and .env created; start script updated. Server printed the Supabase URL on startup.
- **Validation:** Server printed the Supabase URL loaded from .env; git status listed .gitignore and package.json but **not** .env, confirming the key file will not be committed.
- **Decision:** Accepted
- **Reflection:** This was a point again where I did not want to risk ruining everyone else's work, so I decided to follow ai instructions to make sure that I was not doing irreversible damage to our GitHub page. After having spent more time with git and GitHub commands, I became more comfortable with GitHub and understood right away why I the ai would tell me to use a certain command.
- **Responsible Person:** Liam

Entry 1.15 — B5: Installing supabase-js and connecting Express to Supabase

- **Date:** 2026-09-27
- **Purpose of AI Use:** Learning + code generation (Supabase client module).
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "I see .gitignore and package.json and not .env in from git status, I also see the supabase url in the running terminal"
- **AI-Suggested Content:** First commit B4 (remove debug `console.log`; `git add .gitignore package.json`). Then `npm install @supabase/supabase-js`, with a warning that because `node_modules/` is committed in the repo, `git status` will list many `node_modules` changes — commit only `package.json`, `package-lock.json`, and my server files, never `git add ..` Create `server/supabaseClient.js`: reads `SUPABASE_URL` / `SUPABASE_PUBLISHABLE_KEY` from `process.env`, throws a clear error if missing, calls `createClient(url, key, { auth: { persistSession: false, autoRefreshToken: false } })` (server-side settings), and exports it with `module.exports` (compared to a C++ header + `#include`). Import it in `index.js` with `require("./supabaseClient")`. Explained `async/await` (waiting for Supabase's reply over the internet without freezing the server). Temporary test route `GET /api/supabase-test` that attempts a login with fake credentials; expected Supabase reply "Invalid login credentials", which proves the connection and key work.
- **Outcome:** Library installed, `server/supabaseClient.js` created and imported. Test route returned `{"connected":true,"supabaseSaid":"Invalid login credentials"}`.
- **Validation:** Supabase's reply to a deliberately fake login proved the request reached my project and the publishable key was accepted.
- **Decision:** Accepted
- **Reflection:** This was another instance of me not wanting to mess up the repository for my classmates and myself.
- **Responsible Person:** Liam

Entry 1.16 — B6: Implementing POST /api/register with Supabase Auth signUp

- **Date:** 2026-09-27
- **Purpose of AI Use:** Learning + code generation.
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "I got the output: connected true, supabaseSaid "Invalid login credentials""
- **AI-Suggested Content:** Commit B5 (remove test route; `git add package.json package-lock.json server/supabaseClient.js server/index.js`). For development, turn off "Confirm email" in Supabase (Authentication → Sign In / Providers → Email) so accounts work immediately and duplicate emails return an error (note to revisit before release). Updated register route: made `async`; kept validation; calls `supabase.auth.signUp({ email, password, options: { data: { full_name: fullName } } })`; returns 409 "The email you entered is already being used." when `error.code === "user_already_exists"` (matching the 409 check in the teammate's `register.js`), 400 for other Supabase errors (logged with `console.error`), and 201 with the new user's id/email on success (no tokens returned). Explained that Supabase stores users in its own `auth.users` table and hashes passwords, and that full name goes in user metadata. Tests: curl valid registration → 201; repeat → 409; check the user in the Supabase dashboard (Authentication → Users).
- **Outcome:** Registration works: first request returned 201 Created; repeating it returned 409 Conflict; the test user appeared in Supabase → Authentication → Users.
- **Validation:** curl tests for success and duplicate email; confirmed account in the Supabase dashboard.
- **Decision:** Accepted
- **Reflection:** It was at this point that I understood what was happening inside the test commands that I was being provided. I was sending arguments for the server to receive and to inspect what the server would send back.
- **Responsible Person:** Liam

Entry 1.17 — B7: Implementing POST /api/login with Supabase signInWithPassword (written by me with hints)

- **Date:** 2026-09-27
- **Purpose of AI Use:** Learning + guided exercise (I write the code; AI gives concepts, hints, and review).
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "Everything output as expected."
- **AI-Suggested Content:** Commit B6 (`git add server/index.js`). Concepts: `signInWithPassword({ email, password })` returns `{ data, error }`; on error reply **401** with the generic "Invalid email or password." (no account enumeration, per earlier lesson); on success reply **200** with the user's id, email, full name (`data.user.user_metadata.full_name`) and `data.session.access_token`. Explained access tokens (like a wristband proving you already showed your ticket) that the front end will send with later requests (e.g., resume upload). Hints: mirror the register route's structure (`async, await, const { data, error }`, early `return`). Tests: correct password → 200; wrong password → 401; unknown email → 401 with identical message.
 - First attempt (my code): kept the 400 input check; `await supabase.auth.signInWithPassword({ email, password }); if (error.code === "invalid_authentication") → 409 "Invalid Email or Password."`; other errors logged and returned as 400 with `error.message`; success → 200 with id, email, fullName. I reported all tests as expected. AI review: good structure and a good instinct to separate wrong-credential errors from other errors, but (1) Supabase's code for wrong email/password is `invalid_credentials`, not `invalid_authentication`, so that branch never runs and wrong-password/unknown-email requests actually fall through to **400 "Invalid login credentials"** (Supabase's raw message), not 401 — the tests only looked right because the messages are similar; (2) the status should be 401, not 409 (409 = conflict, used for duplicate email); (3) `accessToken: data.session.access_token` missing from the success reply; (4) for unexpected errors, reply 500 with a generic message instead of exposing `error.message`. Lesson: check the status line exactly, and read the server Terminal — the `console.error` output shows the real `code` value.
 - Second attempt: fixed the error code (`invalid_credentials`), status 401, and added `accessToken`. Tests: correct password → 200 OK; wrong password and unknown email → 401 with identical "Invalid email or password." AI review: remaining issues — (1) pasted code contains the typo "psassword" while the test output shows "password", so the file I tested and the code I pasted differ (check the saved file); (2) fix #4 not applied: unexpected errors still return 401 with `error.message` instead of 500 with a generic message; (3) indentation (Format Selection).
- **Outcome:** Login works for success and wrong-credential cases after two review rounds. Final version (third attempt): typo fixed, unexpected errors now return 500 "Something went wrong. Please try again." while still logging details with `console.error`. AI review: logic complete and correct; only remaining polish is indentation.
- **Validation:** I looked at the terminal output and checked that the code that was displayed was the correct one when checking them against the ones in the `index.js` file.

- **Decision:** Accepted
- **Reflection:** All the testing came back correct and at this point, I did not share my output with the ai because I knew what I was looking for to confirm the functioning error calls.
- **Responsible Person:** Liam

Entry 1.18 — B8: Inspecting the classmate's front-end branch before integrating it with my back end

- **Date:** 2026-09-27
- **Purpose of AI Use:** Learning + process guidance (Git collaboration, integration planning).
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "The code is pushed" (B7 login route committed and pushed to `feature/login-backend`).
- **AI-Suggested Content:** Before merging anything, inspect the classmate's `feature/registration-login` branch read-only: `git fetch` (download teammates' branches without changing my files; explained `fetch` vs `pull` and `origin/...` remote branches), `git log --oneline -5 origin/feature/registration-login`, list its files excluding `node_modules`, check whether `client` is still a submodule on that branch (`git ls-tree origin/feature/registration-login client` → `160000 commit = submodule, 040000 tree = normal folder`), and `git diff --stat` between my branch and theirs to predict merge conflicts (especially in `server/index.js` and `package.json`). Plan: then test both together on a temporary integration branch without changing either original branch.
 - Results: `git fetch` downloaded `feature/registration-login`, `AI_log/Arman`, and a newer `origin/master` (65d9366). On the classmate's branch `client` is a normal folder (`040000 tree`, so the submodule problem is fixed there) and it contains `login.html`, `login.js`, `logo.png`, `register.css/html/js`, plus `README.md`, `package.json`, `package-lock.json`, `server/index.js`. The two-dot `git diff --stat` showed many "deletions" (e.g., `documentation/`, `minutes/`, `userStories.md`, `AI_Logs/`, my `server/supabaseClient.js` and `.gitignore`).
 - AI interpretation: a two-dot diff compares the two branch tips directly, so files that exist only on my branch appear as "deleted" — it does not prove the classmate deleted them. Also noticed his branch has commits not in my history and is missing files that exist on master, so it may be based on a different/older starting point. Next checks (still read-only): three-dot diff `origin/master...origin/feature/registration-login` (only what HIS branch changed since it split off), `git merge-base` (do the branches share history at all?), and `git log/git ls-tree` on the new `origin/master` to see whether the repo owner already fixed `client/`.
 - Results: `git merge-base origin/master origin/feature/registration-login` = 79f6402, which is the tip of the classmate's branch; the three-dot diff was empty; new `origin/master` commits include PR #17 (Arman's AI log) and meeting minutes; `client` on master is a normal folder (`040000 tree`).
 - AI interpretation: the classmate's entire branch is already contained in `master` (it was merged), so nothing on his branch would delete files, and `client/` is fixed on master. New plan: bring the latest `master` into my `feature/login-backend` branch rather than merging his branch directly. Preparation before merging: (1) move my old Discord copies of the client files out of the repo (`mv client ~/Desktop/client-discord-backup`) since master's real `client/` files would collide with them; (2) undo the local, uncommitted `node_modules` changes from `npm install` (`git restore node_modules + git clean -fd node_modules`) so the merge isn't blocked — then `npm install` again after merging. Git refuses a merge rather than overwrite uncommitted work, so these steps just clear the way.

- After prep: backup copied to Desktop (4 files); `git status` showed only `deleted: client` (the old submodule pointer's folder was moved). AI: run `git restore client` to put back the empty placeholder so the change doesn't block the merge, then `git merge origin/master --no-edit` (`--no-edit` accepts Git's default merge message instead of opening VS Code). If conflicts appear, paste `git status`; `git merge --abort` safely undoes an in-progress merge.
- **Outcome:** `git merge origin/master --no-edit` succeeded with no conflicts: brought in the classmate's `client/` pages (`login.html`, `login.js`, `logo.png`, `register.*`), Arman's AI log PDF, updated docs/minutes/user stories, and replaced the broken `client` submodule pointer with the real folder (`delete mode 160000 client`). Working tree clean; branch 20 commits ahead of its GitHub copy.
- **Validation:** Read the merge summary and `git status` ("nothing to commit, working tree clean").
- **Validation:** I looked through the GitHub repository and saw that all the files had been merged correctly to my branch.
- **Decision:** Accepted
- **Reflection:** Asking ai for help was to ensure I did not ruined the project, especially after another merge had gone to our master branch before my push.
- **Responsible Person:** Liam

Entry 1.19 — B8e: End-to-end test of my back end with the classmate's register/login pages

- **Date:** 2026-09-27
- **Purpose of AI Use:** Testing guidance + front-end/back-end contract review.
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: pasted successful merge output and clean `git status`.
- **AI-Suggested Content:** `npm install` to restore the Supabase library (ignore the `node_modules` changes it causes; never commit them); `npm start`; in the browser register a new account at `http://localhost:3001/register.html` (valid password, then same email again to see the duplicate message), log in at `/login.html` with correct and wrong passwords, confirm the new user in Supabase → Authentication → Users; share `client/login.js` so the AI can check it matches my API contract (field names, status codes 200/401, `accessToken`); push the merged branch.
 - Follow-up: shared the classmate's `login.js` and said I had permission to capitalize the P's and E's in "password"/"email", which I did. AI review: **JavaScript is case-sensitive** — capitalizing user-visible text (e.g., the message string) is fine, but changing code identifiers (`form.elements.email`, `form.elements.password`, the `email/password` keys sent to the server, variable names) breaks the page, because they must exactly match the HTML `name` attributes and the names my server destructures (`email`, `password`). Other contract findings: (1) `querySelector("#loginForm")` must match the form's `id` in `login.html` (the Discord version used `login-form`) — otherwise `form` is `null` and the script crashes; (2) on success it redirects to `/resume`, which doesn't exist yet (resume page is another teammate's task) → "Cannot GET /resume" is expected for now; (3) the `accessToken` my API returns is not saved — suggest `localStorage.setItem("accessToken", ...)` so the resume upload can prove who is logged in; (4) every failure shows "invalid", including 400/500 cases — could display the server's `error` message instead; (5) no try/catch for network failure. These are suggestions for the front-end owner, not changes to make unilaterally.
 - Results: I only capitalized letters inside message strings. `grep` confirmed `<form id="loginForm">` matches `login.js`. Logging in with correct credentials redirected to "Cannot GET /resume" (expected — proves `/api/login` returned 200 through the real page). Clicking the register page's link to the login page showed "Cannot GET /login".
 - AI diagnosis: the front end uses extension-less URLs (`/login`, `/resume`), but `express.static` only serves `login.html` at `/login.html` by default. Back-end fix (my scope, keeps the front-end code unchanged): `express.static(path.join(__dirname, "..", "client"), { extensions: ["html"] })`, so `/login` → `login.html` and, once it exists, `/resume` → `resume.html`. Tell the resume-page teammate to name the file `resume.html` in `client/`.
- **Outcome:** After adding `{ extensions: ["html"] }`, the register → login link works. End-to-end tests through the real pages all passed (new registration, duplicate email, correct and wrong login), and the new user appeared in Supabase.
- **Validation:** Manual browser testing of all cases + Supabase dashboard check.
- **Decision:** Modified before use

- **Reflection:** I made some modifications to my teammates code in order for us to follow the same punctuation and conditions. I did not follow the ai when the wanted me to proceed in a formal manner because I had discussed with my classmate and was given the ok to proceed.
- **Responsible Person:** Liam

Entry 1.20 — B9: Opening a pull request to merge my back end into master

- **Date:** 2026-09-27
- **Purpose of AI Use:** Process guidance (pull request / code review workflow) + documentation writing (PR description).
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "I tested the outcomes and checked supabase and everything looks good so I am ready to push to main"
- **AI-Suggested Content:** Don't push directly to `master` (the repo's main branch); the course grades the PR + code review workflow. Steps: commit the `express.static` fix (`git add server/index.js` only), `git push`, open a pull request `feature/login-backend` → `master` on GitHub, request a teammate's review, and let the reviewer approve/merge. AI drafted a PR description covering: what changed (middleware, Supabase client, `/api/register`, `/api/login`, extension-less page URLs, `.gitignore`), how to test, setup notes for teammates (run `npm install`; create their own `.env` with `SUPABASE_URL` and `SUPABASE_PUBLISHABLE_KEY`, keys shared privately — never in GitHub/Discord channels), the "Confirm email" setting being off for development, and follow-ups for the front-end (store `accessToken`, `resume.html` name) and team (remove committed `node_modules`). Also: invite teammates to the Supabase project.
 - Follow-up: before opening the PR, I edited message punctuation/capitalization in `login.js`, `register.js` and `index.js`, shared the current `login.js/register.js`, and noted a teammate pushed to `master` 20 minutes earlier. AI review found a **front-end/back-end contract mismatch**: the master version of `register.js` allows passwords of **5–15** characters and accepts `*` as a special character, while my server requires **10–20** characters and doesn't accept `*`. Result: e.g. a 6-char password or one whose only symbol is `*` passes the page but gets a 400 from the server ("Error. Try again."), and 16–20 char passwords are blocked by the page but allowed by the server. Also, Supabase's default minimum password length is 6, so 5-char passwords can never work. Recommendation: agree with the front-end teammate on ONE rule set (suggested: 6–15 characters, a digit, one of `* ! @ # $ % ^ & ( )`), update the server's length check and symbol regex to match, and ask the teammate to change 5 → 6 in `register.js`. Then commit my changes (crediting that front-end message edits were made with permission), `git fetch + git merge origin/master --no-edit` to include the teammate's new master commit, re-test, and push before opening the PR.
- **Outcome:** Prepared to merge my branch onto the master branch
- **Validation:** My git status was up to date and fully functioning.
- **Decision:** Rejected
- **Reflection:** I decided to look at the GitHub page to make sure that there were no new commits in the master branch.
- **Responsible Person:** Liam

Entry 1.21 — Coordination issue: a teammate also implemented the register back end on master

- **Date:** 2026-09-27
- **Purpose of AI Use:** Problem solving / team process (overlapping work, merge planning).
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "I made the changes to the numbers and symbols but another one of my teammates did the register back end and pushed to the master branch."
- **AI-Suggested Content:** Two `/api/register` routes can't coexist (Express uses the first match), and merging now would likely conflict in `server/index.js` and `package.json`. Steps: (1) commit my password-rule alignment first so nothing is lost; (2) inspect master read-only before merging — `git fetch`, `git log --oneline -5 origin/master`, `git diff --stat HEAD...origin/master` (what master changed since my last merge), `git show origin/master:server/index.js` and `git show origin/master:package.json` (view their files without switching branches); (3) ask the teammate which database/Supabase project their route uses, since two separate Supabase projects would split user accounts; (4) then decide as a team which register implementation to keep — to be based on comparing the code, not on who pushed first. Process lesson for Sprint 2: assign each task to one person via GitHub Issues to avoid duplicate work.
 - Follow-ups: "He did it without supabase" / "he doesn't have a system to store the data". AI: a register route that doesn't store accounts means nobody could ever log in, so the Supabase version should be the one kept; framed as a project decision (Supabase is on the team's agreed stack; hashed passwords; accounts persist) and credit the teammate's work. AI drafted a team message proposing this. Merge plan: `git merge origin/master --no-edit`; if `server/index.js` conflicts, resolve in VS Code's conflict view — keep my Supabase register route ("Current Change"), but keep any **other** new code from master (e.g., other routes) — and paste the conflicted section to the AI before finalizing; then `git add server/index.js`, `git commit --no-edit`, re-test, `git push`.
 - My decision: push my work to my branch first so the teammate can review it, and only merge into master once he agrees. AI agreed and adjusted: skip merging master for now; check `git status` (abort any in-progress merge with `git merge --abort`), commit only my files, `git push`, and share the GitHub compare link (`/compare/master...feature/login-backend`) or open a **draft pull request** so he can review and comment; merge master + resolve conflicts after agreement.
 - Chose Option B (draft PR). AI gave step-by-step instructions for creating a draft PR on GitHub and an updated PR description (password rules now 6–15 chars, a digit, one of `* ! @ # $ % ^ & ( )`; note about the overlapping register route and crediting the teammate; request for review).
 - Result: GitHub said "No conflicts with base branch". The teammate agreed I can review his code and merge both so they work together. AI: "no conflicts" only means Git found no overlapping **lines** — his route may be in a different part of `index.js` or in another file, so a clean merge could still leave **two** `/api/register` routes (Express would silently use whichever is registered first). Inspect his changes before merging: `git fetch`, `git log --oneline -5 origin/master`, `git diff --stat HEAD...origin/master -- . '!node_modules'`, and `git`

`diff HEAD...origin/master -- server client` (shows exactly what his side changed since our branches split); alternatively view his commit on GitHub.

- **Outcome:** The teammate removed his register back end and moved to the file-upload back end on his own branch. `git log` showed the only new master commit was "cover page added" (25d0ca8), so there is no overlap with my work. Decision: keep my Supabase register + login and merge through the draft PR.
- **Validation:** Confirmed via `git log --oneline -5 origin/master` and GitHub's "No conflicts with base branch".
- **Decision:** Accepted
- **Reflection:** It is a good thing that I checked the Repository because there had been a push to the master repository just 20 minutes before I was planning to merge my code.
- **Responsible Person:** Liam

Entry 1.22 — Merging the PR into master and syncing locally

- **Date:** 2026-09-27
- **Purpose of AI Use:** Process guidance (PR review/merge workflow).
- **Chat Link or Prompt/Response:** Same conversation as Entry 1.1. Links are not available to individual users of Claude Pro chats created inside a project.
 - Prompt: "My teammate removed their register backend and instead pushed the upload file backend to their branch. I will push my code onto main because the last change I saw from the git log command was: 25d0ca8 (origin/master) cover page added..."
- **AI-Suggested Content:** Merge through the PR on GitHub rather than pushing directly to **master** (the repo's main branch): mark the draft **Ready for review**, have a teammate approve it (Files changed → Review changes → Approve), then **Merge pull request** → Confirm, and optionally delete the branch. Afterwards locally: `git switch master`, `git pull`, `npm install`, `npm start`, and a quick register/login check on master. Remind the upload teammate that `/api/login` returns an **accessToken** his upload route can use to identify the user.
- **Outcome:** The previous push was reversed and I was able to push my code the the master repository without issue.
- **Validation:** I pulled my the main repo after it had been pushed and tested it myself again on the local web server and everything was fully functioning
- **Decision:** Accepted
- **Reflection:** At this point, I was a lot more confident in my abilities to navigate git with terminal commands as well as recognise when certain situation might cause issues even if the ai had not pointed them out. With that said, I was starting this project with very little knowledge on all topics involved so I found it very helpful to have ai guide me and teach me as I worked on the first sprint of the project.
- **Responsible Person:** Liam